

REMOTE CONTROLLED CAR USING BLUETOOTH RC CONTROLLER

A Project Report Submitted in partial fulfillment of the Requirements for the award of the Degree of
BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

By

TARUN RAJGURU 234211

SWAYAM GUPTA 234209

Under the esteemed guidance of

Mr. Dhanraj Jadhav



DEPARTMENT OF INFORMATION TECHNOLOGY

BUNTS SANGHA MUMBAI'S

**ANNA LEELA COLLEGE OF COMMERCE AND SHOBHA JAYARAM SHETTY
COLLEGE FOR BMS**

(Affiliated to University of Mumbai)

MUMBAI 400070 MAHARASHTRA

2025-26

DEPARTMENT OF INFORMATION TECHNOLOGY

Table Of Content

Chapter	Title	Page No.
Chapter 1	Introduction	3
1.1	Background	3
1.2	Objectives	3
1.3	Purpose, Scope, and Applicability	4
1.3.1	Purpose	4
1.3.2	Scope	5
1.3.3	Applicability	5
1.4	Achievements	5
1.5	Organization of Report	6
Chapter 2	Survey of Technologies	7
2.1	Existing System	7
2.2	Proposed System	8
2.3	Requirement Analysis	9
2.4	Hardware Requirement for Hardware Purpose	12
2.5	Software Requirement	14
2.6	Justification of Selection of Technology	16
Chapter 3	Requirements and Analysis	18
3.1	Problem Definition	18
3.2	Requirements Specification	19
3.3	Planning and Scheduling	22
3.4	Software and Hardware Requirements	22
3.5	Preliminary Product Description	25
3.6	Conceptual Models	25
Chapter 4	System Design	33
4.1	Basic Modules	35
4.2	Data Design	35
4.2.1	Schema Design	35
4.2.2	Data Integrity and Constraints	36
4.3	Procedural Design	36
4.3.1	Logic Diagrams	36
4.3.2	Data Structures	37
4.3.3	Algorithms Design	38
4.4	User Interface Design	39
4.5	Security Issues	40
4.6	Test Cases	

CHAPTER 1 : INTRODUCTION

1.1 Background

Bluetooth Remote Controlled Car Project discusses the incorporation of wireless technology into robots. A microcontroller is paired with a mobile device for moving a car with the help of a Bluetooth module in the project.

This is an educational project for the design of a Bluetooth-based RC car with an HC-05 Bluetooth module, L293D motor driver, and an Arduino UNO. Commands sent by a smartphone are processed by an Arduino to drive the car. Real-time control, hardware interfacing, and embedded programming are shown by the project and are thus eminently suitable for students and hobbyists.

1.2 Objectives

- **Wireless Control:** Provide for smooth wireless transmission between the car and either the RC controller or smartphone through Bluetooth (HC-05 or HC-06 module).
- **Integration with Arduino:** Use Arduino (e.g., Uno or Nano) to drive the car's motors based on instructions received via Bluetooth.
- **Motor Driver Interface:** Motor driver interface L298N or L293D to control the speed as well as the direction of the motors.
- **SMOOTH MOVEMENT:** Enjoy smooth movement in all four directions (forward, backward, left, right).
- **User-friendly Operation:** Ensure easy and reliable control using a simple mobile app or Bluetooth joystick/controller.

Accessibility: Make the car usable by utilising easily accessible devices like smartphones to encourage accessibility.

- **Versatility:** Think about potential uses beyond mere movement, such as adding sensors or cameras for more advanced functions.
- **Educational:** Provide a hands-on learning experience for hobbyists or students interested in robotics or embedded systems.

- **Enjoyment and Interaction:** Offer an interesting and engaging experience interacting with robotics and technology. Purpose, Scope, and Applicability

1.3.1 Purpose

Main goal for this project is to design a user-controllable car system via a smartphone to run through Bluetooth signals. The system demonstrates fundamentals of embedded systems as well as wireless communication.

1.3.2 Scope

Takes the Arduino UNO as the primary controller.

- Executes motor control by employing the L293D driver module.
- Receives commands through an Android phone using the HC-05 Bluetooth module.
- Supports different directions: forward, backward, left, right, and stop.

1.3.3 Applicability

This microcontroller-based car project is also highly usable in various applications. It can also be utilized during electronics and embedded systems instructions in schools. It could also serve as a prototype for working home automation systems and for small robot systems. For hobbyists and DIY folks, this is a great opportunity for learning microcontroller interfacing hands-on as well as electronics building, educating them as well as keeping them entertained.

1.4 Achievements

1. Bluetooth Connectivity Established

- ☐ Successfully paired Arduino with a smartphone via a Bluetooth module (e.g., HC-05 or HC-06).

2. Motor Control via Arduino

- Controlled DC motors or servo motors using a motor driver (e.g., L298N or L293D).

3. Wireless Control via Mobile App

- Built or used an app (like MIT App Inventor or a generic Bluetooth terminal) to send commands wirelessly.

4. Custom Mobile App Interface

- Designed a user-friendly interface with buttons or joystick-style controls to operate the car.

5.Full Directional Movement

- Achieved forward, backward, left, right, and stop commands accurately.

6.PWM Speed Control

- Implemented speed variation using Pulse Width Modulation (PWM) signals.

7. Battery Power Management

- Integrated a portable power source (e.g., Li-ion battery) with proper voltage regulation.

8.Chassis Assembly & Design

- Designed and built a custom chassis to mount all components securely.

1.5.Organization of Report

- Chapter1-Introduction – Background, objectives, and scope of the project.
- Chapter2- System Design and Methodology – Diagrams, component interactions, and architecture.
- Chapter3- Technology Stack – Description of tools, components, and platforms used.
- Chapter4- Security Features – Basic security considerations and optional enhancements.

Chapter 2: Survey of Technologies

2.1 Existing System

The existing systems for remote-controlled vehicles commonly use RF (radio frequency) technology or infrared-based remotes for controlling movement. These traditional systems are limited by range, require dedicated remote devices, and often lack flexibility for user customization.

Limitations of the Existing System:

- Limited range (in case of IR systems).
- Signal interference in RF modules.
- Lack of smartphone-based control.
- No feedback mechanism from the car to the controller.
- Bulky remote setups.

Remote-controlled (RC) cars have been popular for decades, primarily relying on Radio Frequency (RF) or Infrared (IR) communication. These systems involve a handheld remote controller transmitting signals to the car, which then interprets the instructions to move in the desired direction.

Types of Existing Systems:

1. RF (Radio Frequency) Controlled Cars

- Use RF modules (typically 27 MHz, 49 MHz, or 2.4 GHz).
- Transmitter and receiver communicate wirelessly over fixed frequencies.
- Widely used in toy cars and hobby-grade RC vehicles.

Limitations:

- Signal interference with other RF devices.
- Limited control customization.
- Requires a separate physical remote.

2. IR (Infrared) Controlled Cars

- Use the IR LED on the remote and the IR receiver on the car.
- Communication is line-of-sight and short-range only.

Limitations:

- Obstacle-sensitive (requires a clear path).
- Very short operational range.
- Limited to basic directional control.

3. Wired Controlled Cars

- The controller is connected to the car via a cable.
- Used mostly in very basic toys.

Limitations:

- Severely limited movement.
- Not suitable for remote operation.
- Outdated and rarely used today.

4. Wi-Fi Controlled Cars (Modern Systems)

- Use ESP8266/ESP32 or similar microcontrollers to connect over Wi-Fi.
- Can be controlled via web-based apps or custom Android/iOS applications.

Limitations:

- More complex and costly.
- Requires internet/router setup or hotspot.
- Not as beginner-friendly for basic educational projects.

Common Drawbacks in Existing Systems

- Fixed hardware controllers – No integration with smartphones.
- Limited range or signal quality – especially in IR-based systems.
- No feedback mechanism – No real-time data or monitoring.
- Less flexibility – Cannot easily modify or add features.

- Complexity in setup – Advanced Wi-Fi systems require networking knowledge.

Why Replacement is Needed

- Users now prefer smartphone-based control for convenience.
- Bluetooth offers better integration with mobile apps.
- Custom features and automation are easier to add using microcontrollers.
- Cost-effective for prototyping and educational use.

2.2 Proposed System (Modules)

The proposed system features a Bluetooth-based, remote-controlled car that can be operated via a smartphone app.

Modules in the Proposed System:

1. Bluetooth Module (HC-05/06): Facilitates wireless communication.
2. Microcontroller (e.g., Arduino Uno): Controls the car based on commands.
3. Motor Driver Module (L298N): Controls the motors' speed and direction.
4. Power Supply: Provides power to the microcontroller and motors.
5. Chassis and Wheels: Mechanical structure for movement.
6. Android Application: Interface for the user to send commands.

The proposed system is a Bluetooth-controlled remote car using a smartphone interface. It integrates embedded hardware and software components to achieve wireless control and smooth manoeuvrability.

1. Bluetooth Module (HC-05/HC-06)

- Acts as the wireless communication interface between the smartphone and the car.
- Operates at a 9600 baud rate for serial communication with the Arduino.
- Range: approximately 10 meters in open space.

- Supports Master-Slave configuration (mostly used as Slave in this project).

2. Arduino Uno (Microcontroller Unit)

- Acts as the brain of the system.
- Receives commands via serial from the Bluetooth module.
- Processes input and send control signals to the motor driver.
- Has digital and analogy I/O pins to connect various modules.

3. Motor Driver Module (L298N)

- Controls the direction and speed of the DC motors.
- Capable of driving two motors simultaneously.
- Uses H-Bridge logic to control forward and backward motion.
- Connected to Arduino and powered by a battery.

4. Power Supply (Battery Pack)

- Supplies power to the Arduino and motors.
- Typically 7.4V Li-ion or 9V batteries are used.
- Voltage regulators may be used to provide a stable 5V to modules.
- Rechargeable, making the car reusable and mobile.

5. Chassis and Wheels

- The structural base of the remote car.
- Includes 2-wheel or 4-wheel configuration depending on use case.
- Light-weight acrylic or metal frame used for durability.
- Mounting space for Arduino, battery, and motor driver.

6. Android Smartphone (User Interface)

- Used to send control signals (Forward, Backward, Left, Right, Stop) via Bluetooth.
- Commands sent as ASCII characters (e.g., 'F', 'B', 'L', 'R', 'S').
- Can be implemented using:
 - Bluetooth Terminal App (prebuilt)
 - MIT App Inventor (custom GUI)
 - Android Studio (advanced apps)

7. Custom Android App

- A user-friendly app developed for specific UI and control functions.
- Touch buttons or gesture-based control.
- Future scalability to include features like voice control or speed variation.

8. LEDs (Status Indicators)

- Indicate system states: Bluetooth connected, command received, battery status.
- Useful for debugging and real-time feedback during operation.

9. On/Off Switch

- For manually powering the system on or off.
- Prevents battery drain when not in use.

10. Heat Sink (for L298N)

- Helps dissipate heat from the motor driver during prolonged usage.
- Increases the durability and stability of the system.

2.3 Requirements Analysis

Description	Advantages	Disadvantages
Bluetooth Communication	Wireless control via smartphone	Limited range (~10 meters)
Arduino Microcontroller	Easy to program, affordable, open-source	Limited memory and processing power
L298N Motor Driver	Can control 2 DC motors easily	Generates heat; not efficient for high currents
Android App Interface	User-friendly and customizable	Requires Android device
DC Motors	Lightweight and easy to use	Not suitable for high torque requirements
Power Supply (Battery)	Makes the car portable and mobile	Requires charging/replacement

2.4 Hardware Requirements

- Arduino Uno Board
- HC-05 Bluetooth Module
- L298N Motor Driver Module
- DC Motors (x2 or x4 depending on chassis)
- Battery Pack (7.4V or 9V recommended)
- Car Chassis and Wheels
- Jumper Wires
- Breadboard (if prototyping)
- Switches (optional for manual control)

2.5 Software Requirements

- Arduino IDE (for writing and uploading code to the microcontroller)
- Bluetooth Terminal App / Custom Android App
- Windows/Linux OS (for development)
- USB Driver for Arduino
- MIT App Inventor (if creating a custom Android app)

2.6 Justification of Selection of Technology

The Bluetooth-based approach offers a cost-effective, easily accessible, and compact solution for remote controlling cars. Using an Arduino microcontroller simplifies development and integration with other modules. Bluetooth modules like HC-05 allow smooth communication with smartphones, eliminating the need for separate remote devices. Additionally, this setup is ideal for learning and prototyping in embedded systems and IoT.

Why this technology chosen?

- Bluetooth is universally available in smartphones.
- Arduino provides ease of development.

- Low cost and power consumption.
- Good for short-range indoor/outdoor use.
- Encourages educational exploration of embedded systems.

The decision to use **Bluetooth-based control** combined with an **Arduino microcontroller** is supported by several strong technical and practical reasons:

1. Wireless Communication Simplicity

- Bluetooth (e.g., HC-05) provides a **reliable, short-range communication** protocol.
- Eliminates the need for extra remote-control hardware.

2. Smartphone Integration

- Most users already own a smartphone with Bluetooth capability.
- Control can be achieved through readily available apps or custom-built Android apps (via MIT App Inventor or Android Studio).

3. Cost Efficiency

- Bluetooth modules and Arduino boards are **low-cost** and easily available in the market.
- Ideal for students and hobbyists working on a budget.

4. Easy Prototyping and Development

- Arduino supports open-source development.
- Large online community support and numerous libraries simplify the implementation.
- Simple wiring and modular design make debugging easy.

5. Educational Purpose

- Introduces users to **embedded systems, wireless communication, and IoT principles**.
- Offers hands-on experience with microcontrollers, motor drivers, and real-world hardware-software interfacing.

6. Portability and Compact Design

- Unlike RF remotes that require additional hardware and separate power, smartphones serve both as controller and interface.
- Makes the overall system lightweight and portable.

7. Flexibility in Control Features

- Bluetooth apps can be customized with various control schemes (buttons, joystick, voice commands).
- Easy to upgrade or add more features like obstacle avoidance, speed control, etc.

8. Energy Efficiency

- Bluetooth consumes relatively **low power** compared to Wi-Fi or RF modules.
- Better battery life for the car in short-range usage.

9. Platform Independence

- The system can work with Android, Windows, or even Linux devices if they support Bluetooth.

10. Safety and Range Limitation

- Since Bluetooth range is limited (~10m), it avoids the risk of long-range interference or uncontrolled operation.
- Useful in controlled environments like robotics labs or small demos.

CHAPTER 3: REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

Traditional RC cars employ radio frequency (RF) modules and specially designed controllers with restricted flexibility and exorbitant expenses. Such systems are limited to exclusive remotes with restricted control ranges and are not smartphone-compatible. There is the need for contemporary, user-friendly, and affordable technology. With the implementation of Bluetooth-based technology along with mobile applications, users can operate the RC car through their smartphones with greater accessibility and mobility as well as greater user-friendliness.

Specific Problems to Solve

1. **Wireless command reception**
Ensure reliable Bluetooth communication with a paired smartphone or controller over a range of roughly 5–10 m.
2. **Command parsing & mapping**
Interpret incoming commands and translate them into safe motor control signals, including both direction and speed.
3. **Motor control**
Drive DC or small geared motors smoothly using an appropriate driver, with PWM for speed control and dedicated pins for direction.
4. **Power management**
Deliver stable power to the microcontroller, Bluetooth module, and motors; choose a suitable battery and implement protection circuitry.
5. **Safety & robustness**
Prevent runaway motion by handling lost connections, guard against short circuits, and include an emergency stop function.
6. **User interface**
Design a simple mobile UI—such as buttons or a joystick—or a text-based command format that’s intuitive and minimizes accidental actions.
7. **Physical design**
Build a chassis and wheel assembly capable of carrying the required payload while maintaining good traction and balance.

Context & Motivation

Many hobbyists and students are looking for an accessible platform to explore embedded systems, wireless communication, and motor control. Traditional RC setups typically require

dedicated remote controls, which can add cost and complexity. By using Bluetooth, the project becomes compatible with smartphones, reducing component expenses while enabling a straightforward software interface. This approach allows users to experiment with control logic, user interfaces, and feedback mechanisms in a more flexible, modern, and cost-effective way.

3.2 Requirements Specification

1. Functional Requirements

These define what the system should accomplish.

1.1 Bluetooth Communication

- The car must establish a wireless link with a smartphone via a Bluetooth module (e.g., HC-05 or HC-06).
- The system should reliably receive movement commands such as Forward, Backward, Left, Right, and Stop.

1.2 Motor Control

- The microcontroller must process incoming commands and control the DC motors through a motor driver (e.g., L298N).
- Motor speed should be regulated using PWM (Pulse Width Modulation) signals.

1.3 Direction Control

- The system should turn left or right using differential motor speed control.
- Direction changes should be smooth, avoiding sudden jerks.

1.4 Power Management

- The car should run on a rechargeable battery.
- The system must protect electronics from damage due to low voltage.

1.5 User Interface

- A mobile application (or a serial Bluetooth terminal) should provide on-screen controls such as buttons or sliders for movement.
- The interface should display the Bluetooth connection status.

2. Non-Functional Requirements

These define the system's quality attributes and operational standards.

2.1 Performance

- Command response time must be less than 200 ms.

- The system should operate continuously for at least 30 minutes on a full battery charge.

2.2 Reliability

- Maintain a stable Bluetooth connection within a range of at least 5–10 meters.
- Recover automatically from temporary disconnections without user intervention.

2.3 Usability

- Provide a control interface that is simple and intuitive.
- Ensure the system can be operated by users without technical expertise.

2.4 Portability

- The car should be lightweight and compact for easy handling and transport.

2.5 Safety

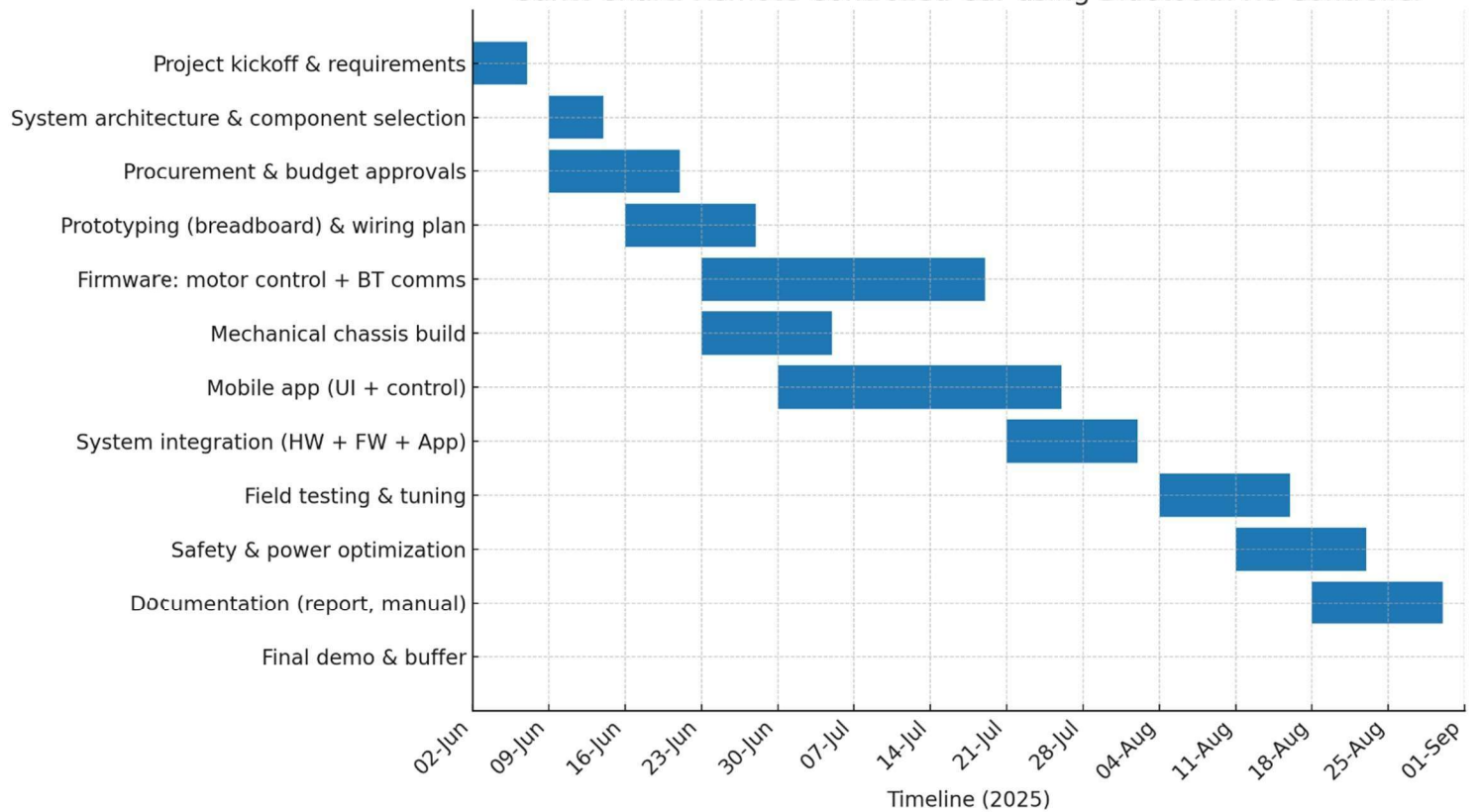
- Protect electronic circuits from short circuits and reverse polarity.
- Automatically stop the motors if the Bluetooth connection is lost.

3.3 Planning and Scheduling

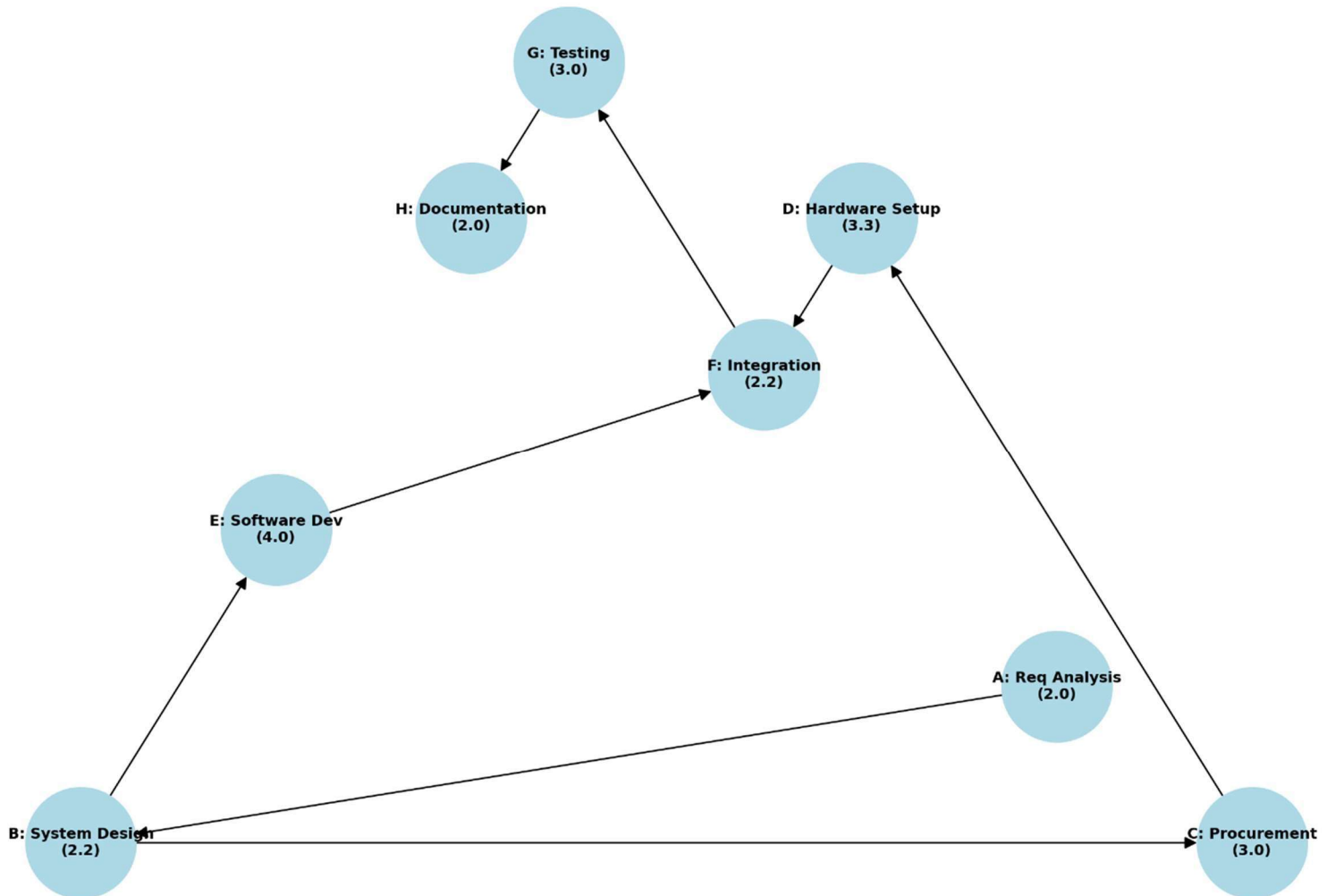
The development will be carried out in five phases:

Phase	Duration	Activities
Requirement Analysis	1 week	Problem definition, requirement gathering
Design	1 week	System architecture, circuit, and app UI
Implementation	2 weeks	Coding microcontroller and mobile app
Testing	1 week	Unit testing and integration testing
Finalization	1 week	Debugging, documentation, final demo

Gantt Chart: Remote Controlled Car using Bluetooth RC Controller



PERT CHART



Task	Task Name	O	M	P	TE = (O + 4M + P)/6	SD = (P - O)/6
A	Requirement Analysis	2	4	6	$(2 + 4 \times 4 + 6)/6 = 4.0$	$(6 - 2)/6 = 0.67$
B	Component Selection	2	3	5	$(2 + 4 \times 3 + 5)/6 = 3.17$	$(5 - 2)/6 = 0.50$
C	Circuit Design	3	4	6	$(3 + 4 \times 4 + 6)/6 = 4.17$	$(6 - 3)/6 = 0.50$
D	Microcontroller Setup	4	6	9	$(4 + 4 \times 6 + 9)/6 = 6.17$	$(9 - 4)/6 = 0.83$
E	Bluetooth Integration	3	5	8	$(3 + 4 \times 5 + 8)/6 = 5.17$	$(8 - 3)/6 = 0.83$
F	Motor Driver Integration	3	5	7	$(3 + 4 \times 5 + 7)/6 = 5.00$	$(7 - 3)/6 = 0.67$
G	App Interface Development	5	7	10	$(5 + 4 \times 7 + 10)/6 = 7.17$	$(10 - 5)/6 = 0.83$
H	Connectivity Testing	2	3	5	$(2 + 4 \times 3 + 5)/6 = 3.17$	$(5 - 2)/6 = 0.50$
I	Range and Signal Testing	2	3	6	$(2 + 4 \times 3 + 6)/6 = 3.33$	$(6 - 2)/6 = 0.67$
J	Motor Ctrl & Safety Testing	3	4	6	$(3 + 4 \times 4 + 6)/6 = 4.17$	$(6 - 3)/6 = 0.50$
K	Debugging and Improvements	4	6	9	$(4 + 4 \times 6 + 9)/6 = 6.17$	$(9 - 4)/6 = 0.83$
L	Demo Presentation	1	2	3	$(1 + 4 \times 2 + 3)/6 = 2.0$	$(3 - 1)/6 = 0.33$
M	User Feedback	2	3	4	$(2 + 4 \times 3 + 4)/6 = 3.0$	$(4 - 2)/6 = 0.33$
N	Minor Bug Fixes	2	3	5	$(2 + 4 \times 3 + 5)/6 = 3.17$	$(5 - 2)/6 = 0.50$

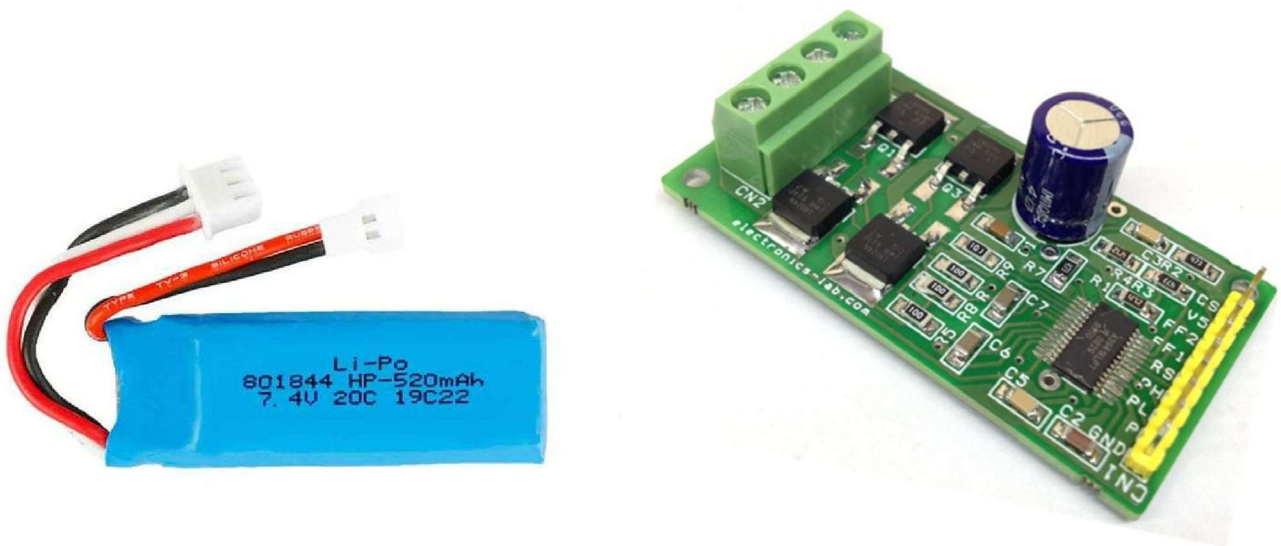
3.4 Software and Hardware Requirements

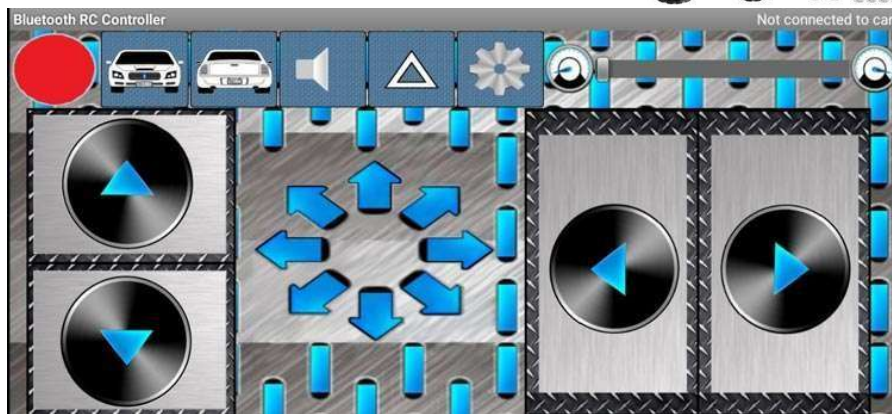
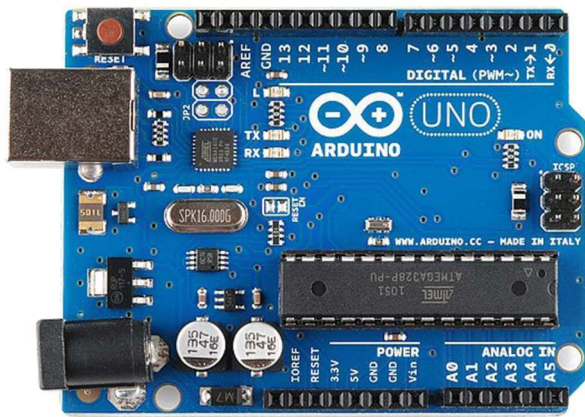
Hardware Requirements

- Bluetooth Module (HC-05 or HC-06)
- Microcontroller (Arduino UNO or similar)
- Motor Driver (L298N or similar)
- DC Motors (2 or 4 depending on design) □ Chassis, Wheels, Battery (Li-ion), Wires

Software Requirements

- Arduino IDE (for programming microcontroller)
- MIT App Inventor / Android Studio (for app development)
- Windows/Linux OS for development environment
- Bluetooth serial communication libraries





3.5 Preliminary Product Description

Preliminary Product Description

The Remote-Controlled Car using a Bluetooth RC Controller is a compact, wireless vehicle designed for control via a smartphone or dedicated Bluetooth controller. It integrates a microcontroller, motor driver, Bluetooth communication module, and rechargeable battery to enable real-time wireless operation.

The vehicle responds to directional commands—forward, backward, left, right, and stop—transmitted over Bluetooth from the user's device. The microcontroller processes these commands and drives the DC motors through a motor driver circuit, ensuring smooth movement and precise steering. Speed control is achieved through Pulse Width Modulation (PWM) signals, allowing fine adjustments to motor speed.

The design emphasizes ease of use, portability, and cost efficiency. With a Bluetooth range of approximately 5–10 meters, it provides reliable operation within line-of-sight. Its lightweight yet robust chassis supports operation on smooth indoor surfaces as well as moderately rough outdoor terrain.

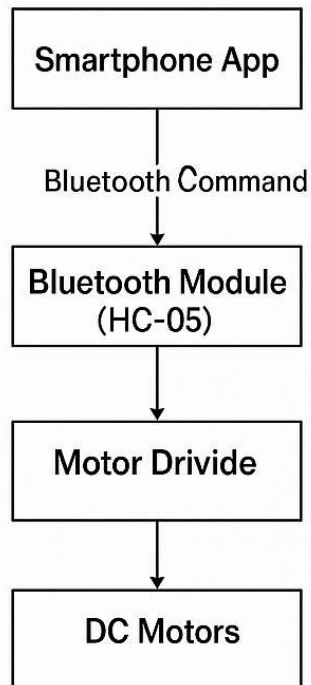
This system is ideal for educational projects, hobbyist experiments, and prototype development in embedded systems and robotics. It offers a hands-on way to explore wireless communication, motor control systems, and customizable RC platform design.

Key Features

- Wireless control via Bluetooth (Android app or dedicated controller)
- Simple, intuitive user interface for command input
- Compact, lightweight chassis for high mobility
- Rechargeable battery for repeated, convenient use
- Motor driver for smooth speed and direction control
- Affordable, customizable design suitable for various applications

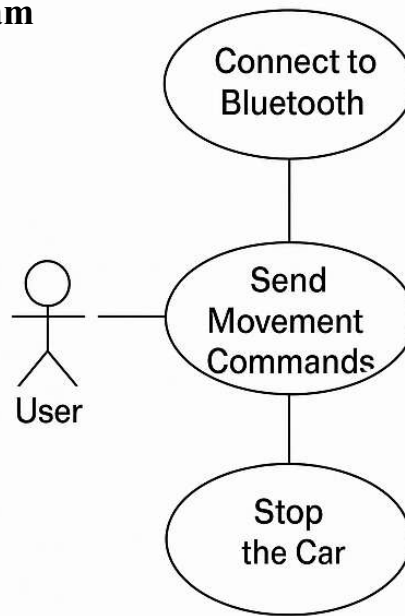
3.6 Conceptual Models of Remote-Controlled Car Using Bluetooth RC Controller

Block Diagram Model

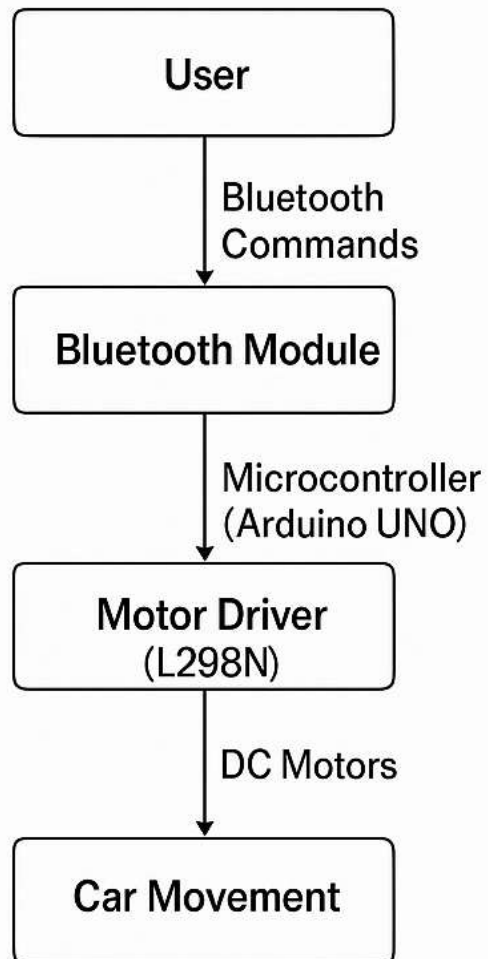


Block Diagram

Use Case Diagram

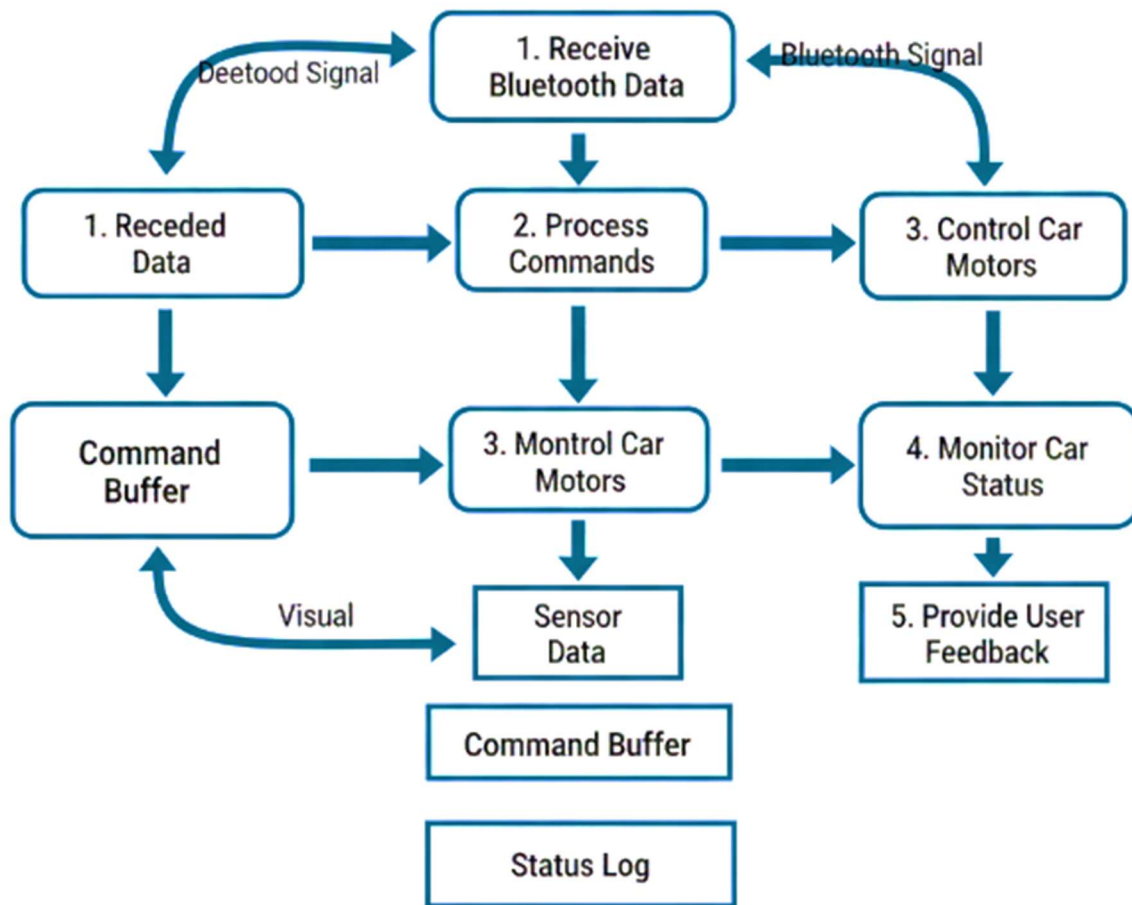


Data Flow Diagram (Level 0)

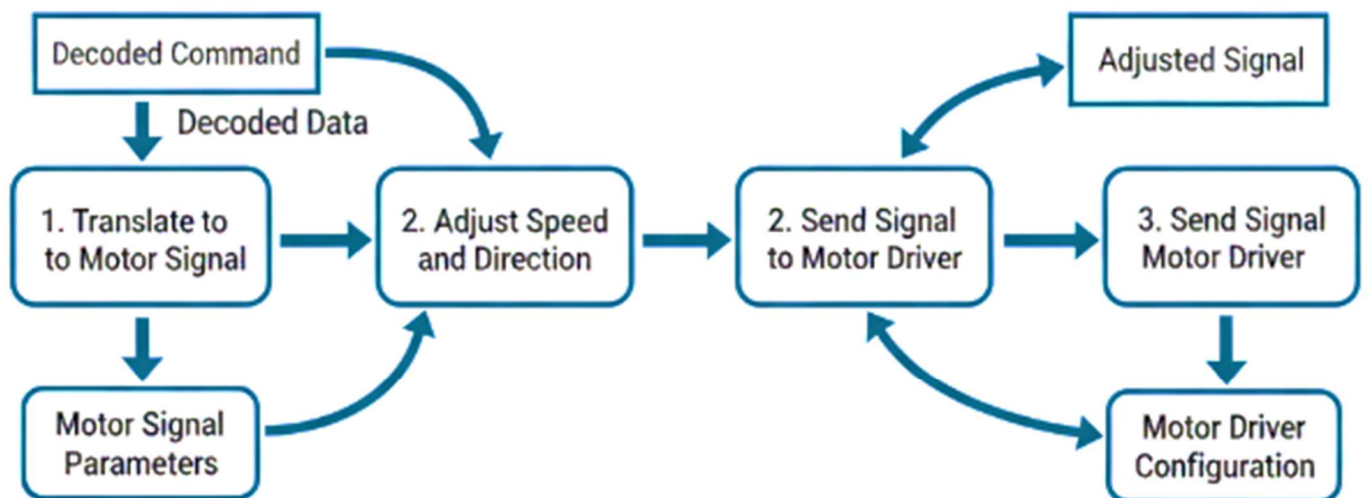


Data Flow Diagram (Level 0)

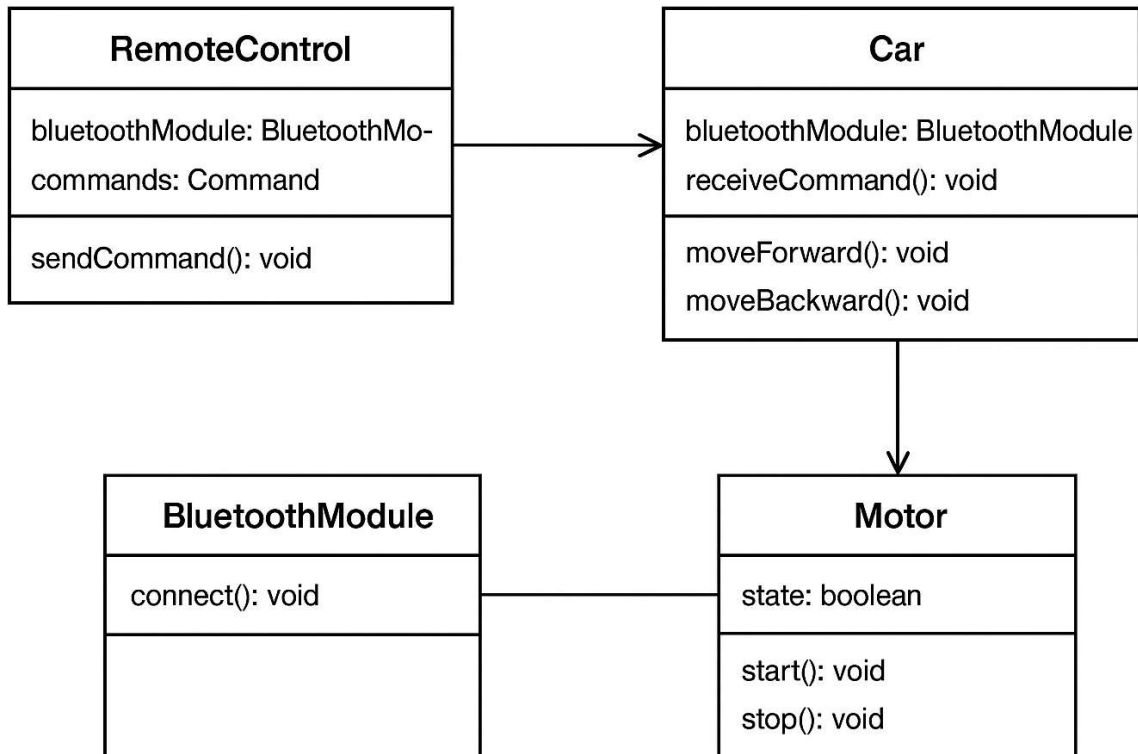
Data Flow Diagram (Level 1)



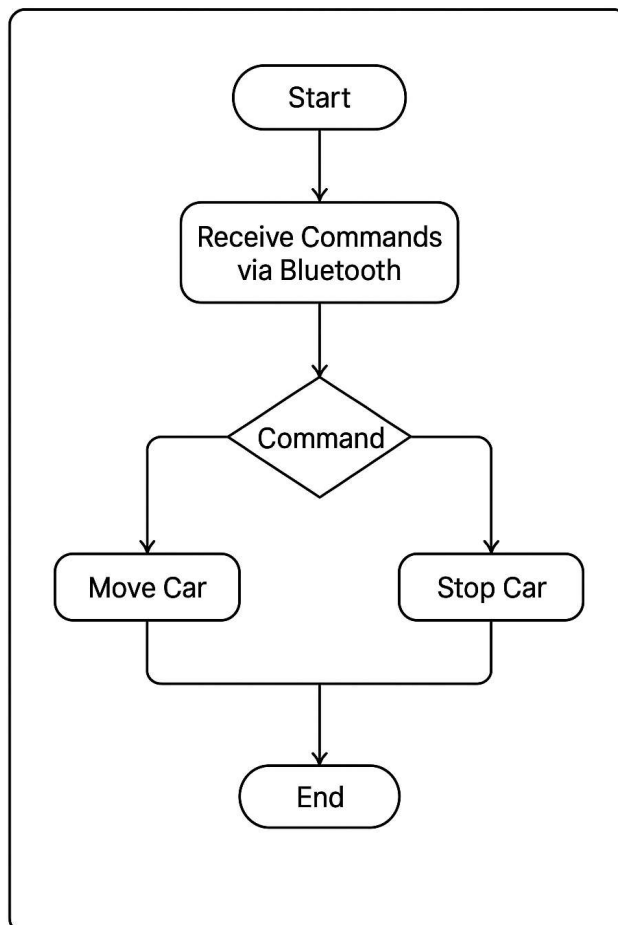
Data Flow Diagram (Level 2)



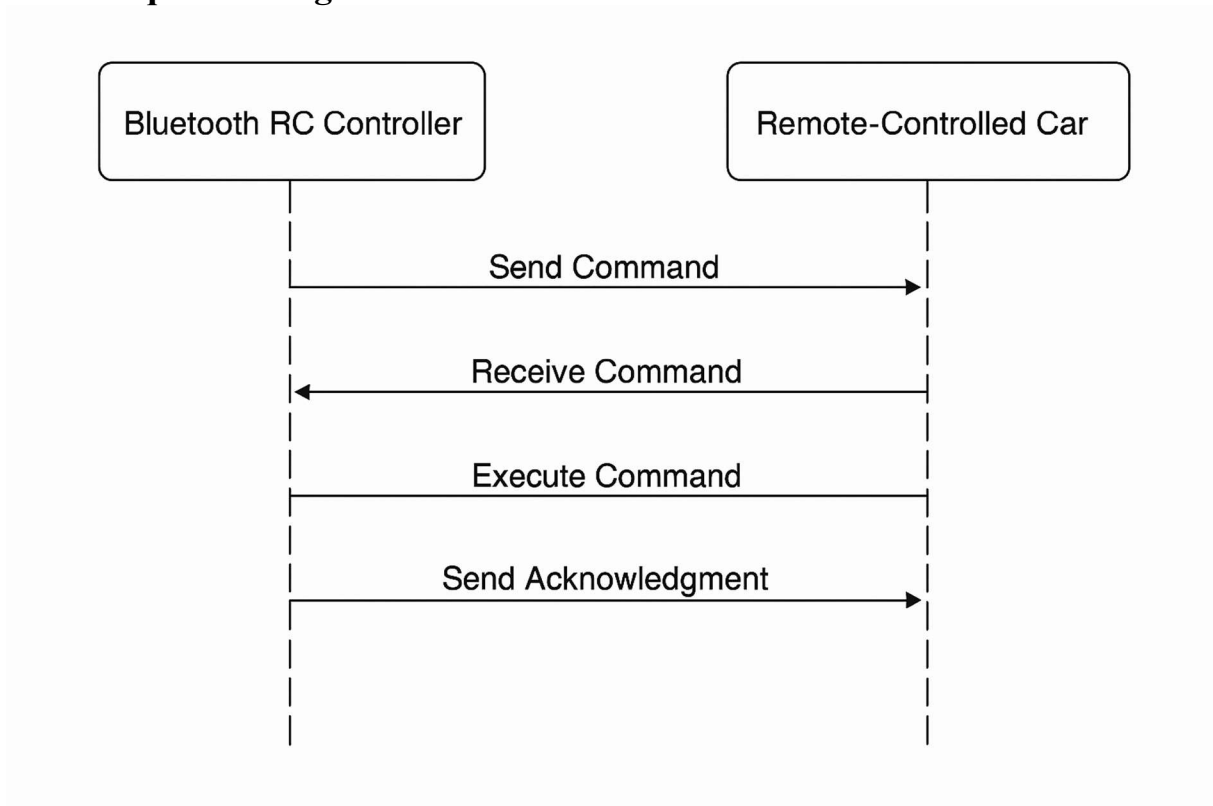
Class Diagram



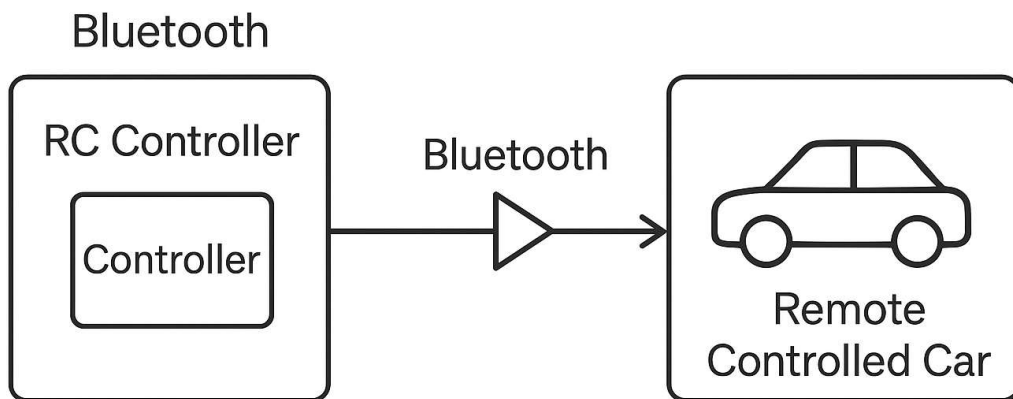
Activity Diagram



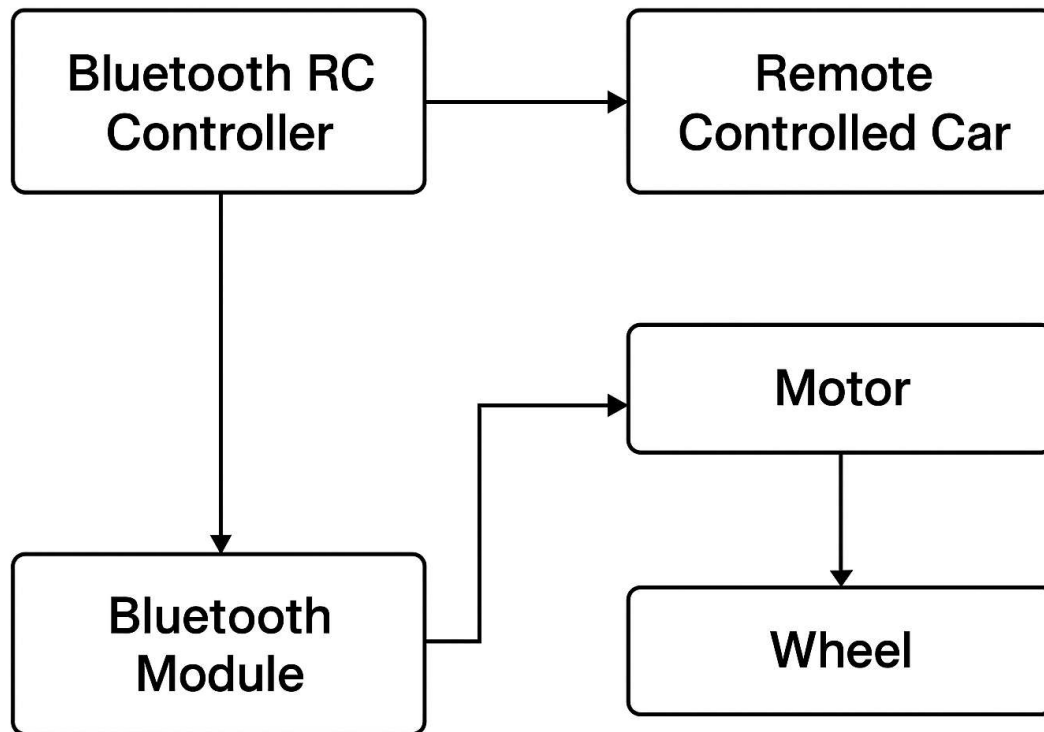
Sequence Diagram



Collaboration Diagram



Object Diagram



CHAPTER 4 - SYSTEM DESIGN

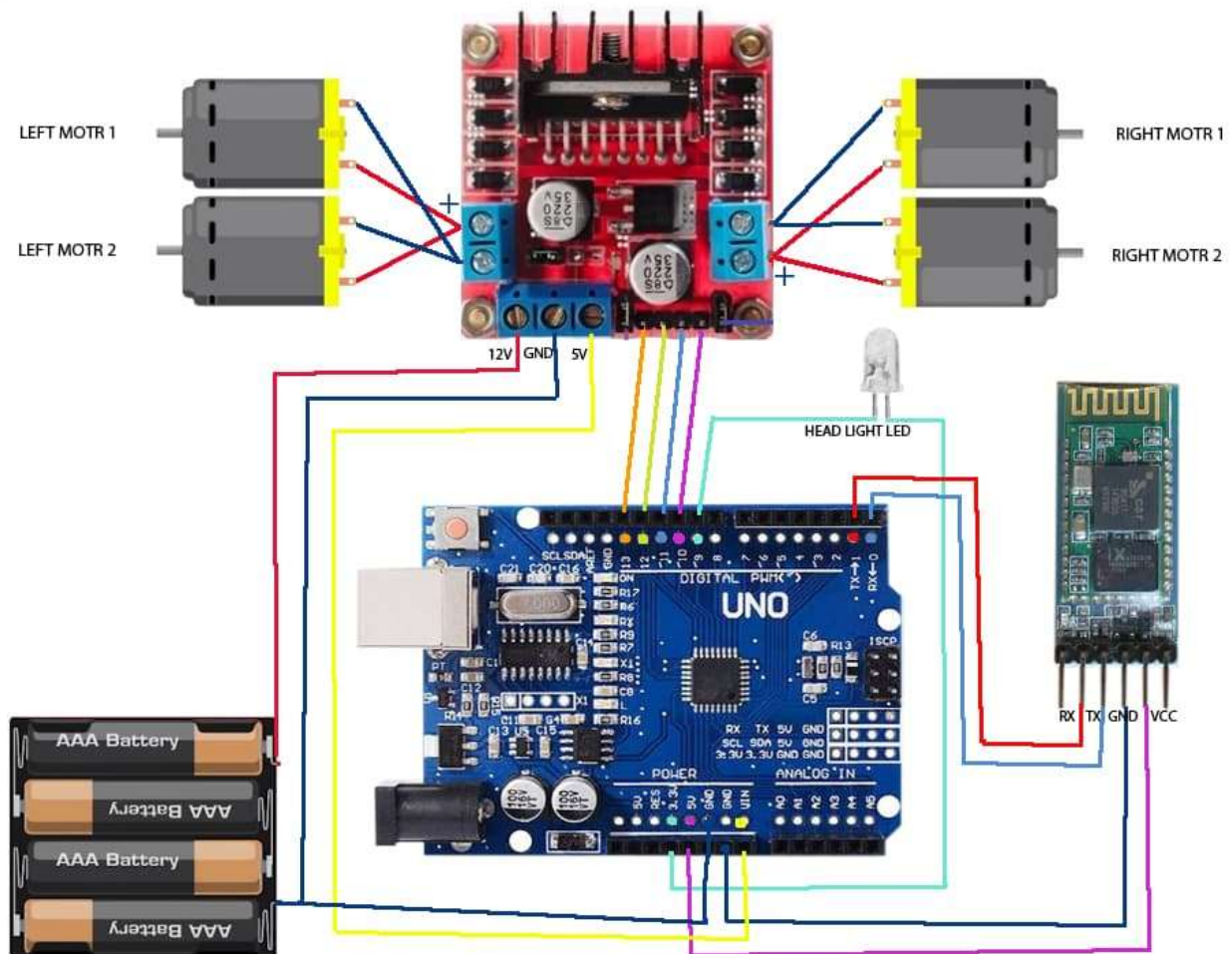
4.1 Basic Modules

It consists of a number of simple modules, and each module does a given task so that the car controlled by the Bluetooth will work efficiently. They include:

1. Power Supply Module
 - o Powers the microcontroller, motor driver, and motors reliably with DC power supply.
 - o Generally consists of a rechargeable battery, voltage controller, and protection circuits..
2. Bluetooth Communication Module
 - o It receives wireless commands through the smartphone or Bluetooth controller.
 - o Normally uses modules like HC-05/HC-06 for serial communication with the microcontroller.
3. Microcontroller Module
 - It is the brain of the system. o
 - Decodes the Bluetooth commands and issues proper control signals to the motor driver.
Examples: Arduino, PIC, or any embedded microcontroller.
4. Motor Driver Module
 - o Acts as an interface between the low-power microcontroller signals and the highpower DC motors.
 - o Provides directional control (forward, reverse, left, right) using H-Bridge circuits (e.g., L293D, L298N).
5. DC Motor & Chassis Module
 - o Includes the motors, wheels, and chassis frame of the car. o Converts electrical control signals into mechanical motion for movement in different directions.
6. Smartphone/Controller Module

- A mobile application or dedicated Bluetooth remote is used to send commands such as Forward, Backward, Left, Right, Stop.
- Provides the user interface for real-time control.

4.2 Data Design



The data design defines how data is stored, transmitted, and processed. Data flow is lightweight and real-time.

4.2.1 Schema Design

Since this project is embedded and lightweight, schema design is minimal:

- User Commands Table (logical structure)

- o
 - o Command_ID
 - o Command_Type (Forward/Backward/Left/Right/Stop)
 - o Timestamp
- System Logs Table (optional for testing/debugging)
 - o Log_ID
 - o Event_Type
 - o Status_Message

4.2.2 Data Integrity and Constraints

Data integrity ensures that the commands and signals exchanged between the Bluetooth controller (smartphone or RC controller) and the car's microcontroller remain accurate, reliable, and consistent. In this project, it is critical to prevent incorrect or corrupted data from causing wrong actions (e.g., car moving forward when the command was to stop).

1. Accuracy of Commands

- o Ensures the correct mapping of user input (e.g., "Forward" button) to the intended car movement.
- o Validates that only valid command codes (e.g., F, B, L, R, S) are executed.

2. Consistency of Communication

Bluetooth data packets should remain consistent during transmission without loss or duplication.

Ensures uniform speed and direction control for smooth operation.

3. Error Detection

- Use checksum or simple acknowledgment to ensure data is received correctly.
- If corrupted, the system can ignore the faulty command and maintain the last safe state.

4. Synchronization

- o Commands must be executed in the correct order to avoid conflicts (e.g., not allowing forward and backward signals at the same time).

5. Reliability

- o
- o If communication is lost, the car should stop automatically instead of continuing with the last command.

Constraints

Constraints define the limits and rules of the system to ensure safe and effective operation.

1. Hardware Constraints
 - o Bluetooth Range: Typically limited to 5–10 meters for HC-05/HC-06 modules.
 - o Battery Capacity: Determines how long the car can operate before recharging.
 - o Motor Driver Limits: Current and voltage ratings restrict motor size and load.
2. Software Constraints
 - o The microcontroller can process only predefined commands (no arbitrary input).
 - o Command structure must follow a fixed protocol (e.g., single-character or string commands).
 - o Processing speed is limited by the microcontroller's clock rate.
3. Safety Constraints
 - o Car must stop when no command is received (fail-safe).
 - o Commands outside the valid set should be ignored.
 - o Overheating or overcurrent protection for motors.
4. Operational Constraints
 - The car cannot operate if the Bluetooth connection is not established.
 - Only one controller can be paired at a time to avoid conflicts.
 - Works effectively only on smooth surfaces (depending on wheel design and motor power).

4.3 Procedural Design

The system is designed as a sequence of procedures: receiving commands, interpreting, and executing motor actions.

4.3.1 Logic Diagrams

Bluetooth remote-controlled cars typically operate using three main components: a Bluetooth remote controller, a Bluetooth receiver module, and the car's control circuit. The logic behind their interaction is fairly straightforward.

The Remote Controller

The remote controller is what you hold to control the car. It contains:

- **Input Devices:** These are the buttons, joysticks, or sliders that a user manipulates. Each input corresponds to a specific command, such as "move forward," "turn left," or "increase speed."
- **Microcontroller:** This is the "brain" of the remote. It reads the input from the joysticks or buttons and translates these physical movements into digital signals.
- **Bluetooth Transmitter Module:** The microcontroller sends the digital signals to this module. The module then packages the data into Bluetooth packets and transmits them wirelessly to the car. This data often includes a unique identifier for the car and the specific command (e.g., Car_ID_123:Fwd for "move forward").

The Car's Control Circuit

This is the part of the system that receives the commands and makes the car move. It consists of:

- **Bluetooth Receiver Module:** This module listens for incoming Bluetooth signals. When it receives a packet addressed to its unique identifier, it accepts the data and sends it to the car's microcontroller.
- **Microcontroller:** This is the car's "brain." It receives the command from the Bluetooth module (e.g., Fwd.). It then decodes this command and sends the appropriate electrical signals to the motor drivers.

- **Motor Drivers:** These are electronic circuits that amplify the low-power signals from the microcontroller and provide enough power to turn the motors. For a remote-controlled car, you typically need at least two motor drivers: one for the drive motors (for forward/reverse movement) and one for the steering servo (for turning).

Logic Diagram

The following diagram illustrates the flow of information:

1. **User Input:** The user presses a button or moves a joystick on the remote controller.
2. **Signal Conversion:** The remote's microcontroller converts the physical input into a digital command.
3. **Wireless Transmission:** The remote's Bluetooth module transmits the digital command wirelessly.
4. **Wireless Reception:** The car's Bluetooth module receives the command.
5. **Command Decoding:** The car's microcontroller decodes the received command.
6. **Motor Activation:** The microcontroller sends the appropriate signals to the motor drivers.
7. **Physical Movement:** The motor drivers provide power to the motors, causing the car to move or turn.

4.3.2 Data Structures

- **Command Queue (Array/List):** Stores incoming commands.
- **Lookup Table (Dictionary):** Maps commands to motor driver pins.
- **Status Flags (Boolean variables):** Connection status, battery status, etc.

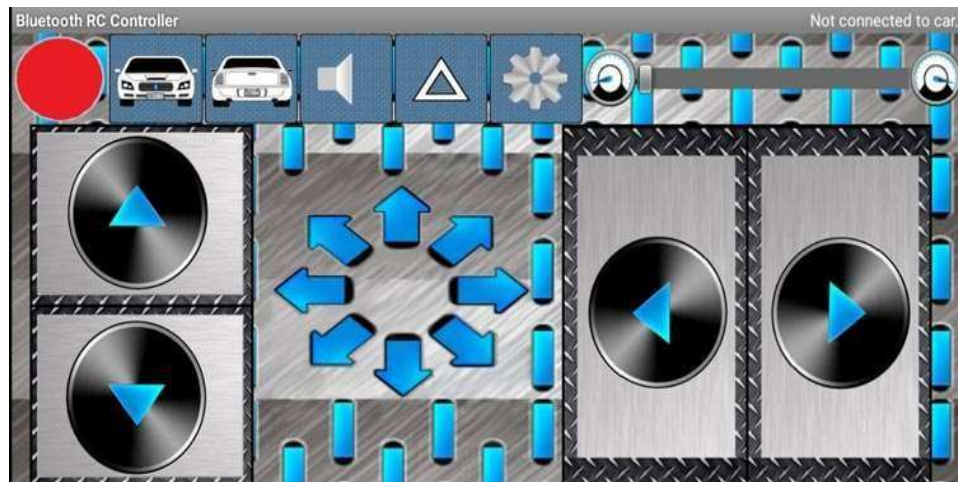
4.3.3 Algorithm Design

Algorithm for command execution:

1. Initialize Bluetooth and motor driver.
2. Wait for incoming command.
3. If command received:

- o Validate command.
 - o Map command to motor control logic.
 - o Send signals to motor driver.
4. Repeat until system shutdown.

4.4 User Interface Design



Mobile App UI :

- o Buttons for Forward, Backward, Left, Right, Stop.
- o Bluetooth connection status indicator.
- Controller UI:
 - o Simple directional keys for navigation.
- Feedback: LED indicator on the car for power and Bluetooth connection.

4.5 Security Issues

1. Unauthorized Access (Pairing Issues)

- If the Bluetooth module (like HC-05/HC-06) does not use proper authentication, any nearby device can connect.
- Hackers could gain control of the car without the owner's permission.

2. Weak or No Encryption

- Most basic Bluetooth modules transmit data in plain text.
- Attackers can intercept commands (forward, stop, turn) and replay them to control the car.

3. Signal Interference / Jamming

- Bluetooth works in the 2.4 GHz band, which is crowded with Wi-Fi and other devices.
- Attackers could jam the signal, making the car unresponsive or uncontrollable.

4. Replay Attacks

- Since commands are often simple (like F, B, L, R), attackers can record and resend them later.
- This allows unauthorized control even without active access.

5. Limited Range Exploitation

- Bluetooth has a limited range (~10 m for classic modules).
- But with special equipment (amplified antennas), attackers could control the car from longer distances than intended.

6. Firmware/Module Vulnerabilities

- Cheap Bluetooth modules often lack updates or secure firmware.
- Malicious devices can exploit known flaws to take control.

7. Denial of Service (DoS) Attacks

- Sending continuous or malformed data packets may overload the microcontroller.
- This could crash the system, stopping the car abruptly.

8. Data Privacy Issues

- If the car is upgraded with features like GPS tracking or cameras, unprotected Bluetooth can leak location or video data.

4.6 Test Cases Design

Sample test cases:

Test Case ID	Description	Input	Expected Output	Result
TC01	Forward movement	Press “Forward”	Car moves forward	Pass/Fail
TC02	Stop command	Press “Stop”	Car halts immediately	Pass/Fail
TC03	Left turn	Press “Left”	Car turns left	Pass/Fail
Test Case ID	Description	Input	Expected Output	Result
TC04	Connection test	Try to connect unpaired device	Connection denied	Pass/Fail
TC05	Low battery scenario	Battery < threshold	Warning/Stop safely	Pass/Fail